Solution Design Document, Version 2.0

Amrit Water WhatsApp AI Sales & Order Agent

Document	Solution Design v2.0
Project	Amrit Water WhatsApp AI Agent
Prepared by	Saloni Sisodiya, AI Solutions Consultant
Date	November 2026
Status	Proof of concept. Demo rates and test data.
Related	BRD v2.0, Test Report v2.0, Runbook v2.0, system prompt in prompts/system-prompt.md

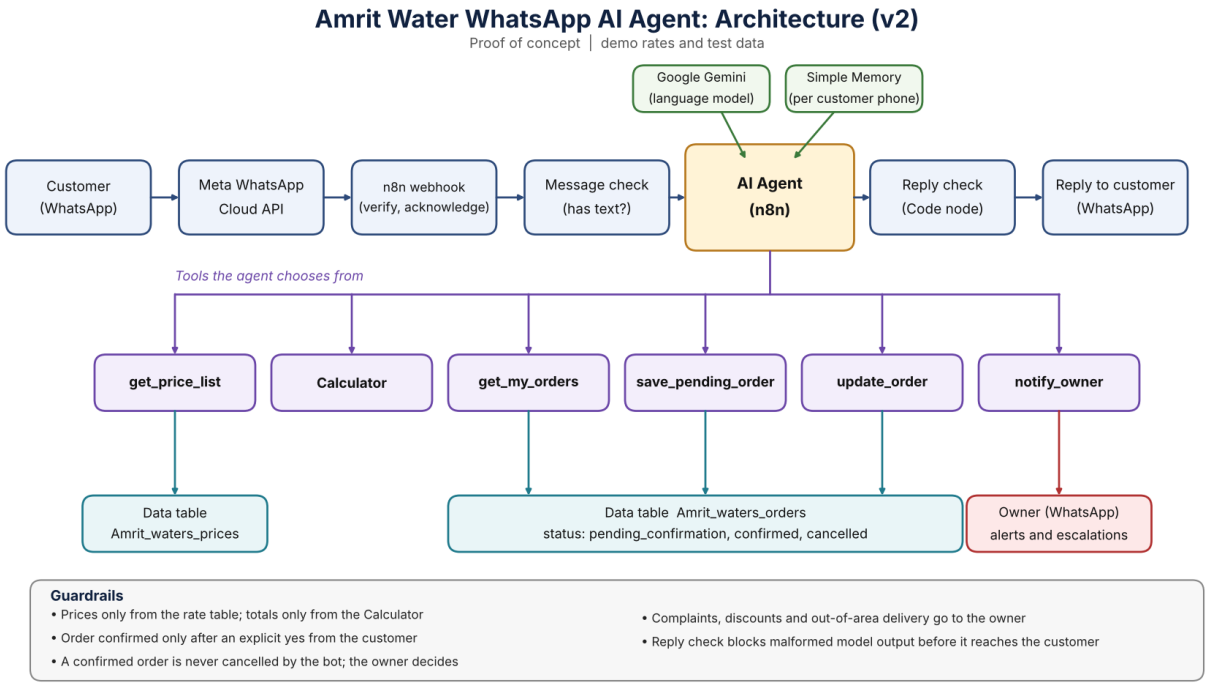
1. Overview

The solution is a tool-using AI agent that runs inside an n8n workflow and talks to customers through the WhatsApp Business Cloud API. Instead of following a fixed pipeline, the agent reads each message, decides which tool to call (price lookup, calculator, order lookup, order save, order update, owner alert), and replies. Business rules, data tables and a reply check keep it within safe limits, and the owner stays in the loop for anything sensitive.

2. Key design decisions

Decision	Choice	Alternatives considered	Reason
Architecture	One agent with tools	Keyword chatbot; fixed-flow assistant (v1); multi-agent	Handles edits, follow-ups and escalation without a rigid script; simpler to debug and hand over than multi-agent
Orchestration	n8n (no-code)	Custom code; managed commerce platform	Visual workflow the client can read; fits a small budget; quick iteration
Model	Google Gemini chat model	Groq-hosted open model (used in early testing)	The Groq free tier hit rate limits and leaked reasoning text into one reply; Gemini was stable in final testing
Pricing source	Rate table plus Calculator tool	Prices inside the prompt; model arithmetic	Rates change without editing the prompt; a wrong total was found when the model did its own maths
Order storage	n8n Data Tables	Google Sheets; external database	Built into n8n, no extra credentials; easy to export later
Memory	Simple Memory keyed by phone number	Database-backed memory	Enough for a proof of concept; resets on restart
Human in the loop	Owner alerts and escalation rules	Fully automatic handling	Money, complaints and cancellations need a person

3. Architecture



Message flow

1. The customer sends a WhatsApp message to the business number.
2. Meta calls the n8n webhook. The workflow separates Meta's verification handshake from real messages, checks the verify token, and acknowledges the call quickly.
3. The workflow extracts the sender's number, the message text and the message id, and continues only for text messages.
4. The AI Agent loads that customer's memory, reads the message, and calls the tools it needs.
5. A reply check inspects the agent's output. Malformed or overlong output is replaced with a safe fallback message.
6. The reply is sent to the customer. Alerts for the owner are sent by the notify_owner tool during step 4.

4. Workflow components

Node	Type	Purpose
WhatsApp Webhook	Webhook (GET and POST)	Receives Meta's verification requests and message events
Is Verification Request	If	Separates Meta's verification handshake from message events
Verify Token Matches	If	Compares the received verify token with the secret
Return hub.challenge / Return 403 Forbidden	Respond to Webhook	Completes verification, or rejects a wrong token
Process Incoming Message	Edit Fields	Extracts sender number, message text and message id
Acknowledge Message	Respond to Webhook	Returns success to Meta immediately
has message text	If	Lets only text messages reach the agent
AI Agent	n8n AI Agent	Runs the model with memory and tools under the system prompt
Code in JavaScript (reply check)	Code	Replaces malformed or overlong model output with a fallback message
send message to customer	WhatsApp Business Cloud	Sends the reply to the sender

5. Tool specifications

Tool	Implementation	Inputs	Used when
get_price_list	Data Table: get all rows from Amrit_waters_prices	None	The customer asks a price, or a total is needed
Calculator	n8n Calculator tool	Arithmetic expression	Before stating any total
get_my_orders	Data Table: get rows from Amrit_waters_orders filtered by the sender's number	Sender number (fixed by the workflow)	Status questions, repeat orders, finding an open draft
save_pending_order	Data Table: insert row	Bottle size, cartons, delivery location, customisation (from the model). Phone number and status pending_confirmation are fixed	All details are known and the summary is ready to confirm
update_order	Data Table: update row by id	Order id; cartons, location, customisation, status	Editing a draft, confirming after an explicit yes, cancelling a draft
notify_owner	WhatsApp Business Cloud: send message	Message text (from the model). Recipient is the owner's number, fixed	Confirmed order, payment method, complaint, discount, out-of-area delivery, cancellation request for a confirmed order

6. Data model

Table	Columns	Notes
Amrit_waters_orders	id (automatic), phone_number, bottle_size, quantity_cartons, delivery_location, customisation, status	Status values: pending_confirmation, confirmed, cancelled. One draft per order, edited in place
Amrit_waters_prices	id (automatic), min_cartons, price_per_carton	Demo rows: 1 to ₹108, 10 to ₹102, 50 to ₹96. The highest slab at or below the order quantity applies

7. Agent design

System prompt structure

The full prompt is stored in prompts/system-prompt.md. It is organised in this order, with the earliest items taking priority:

- Output rule: reply with only the final customer message; never write reasoning, plans or tool names.
- Language rule: judge the language only from the customer's latest message and reply in English or Hinglish accordingly.
- Role, and the customer's phone number as a variable supplied by the workflow.
- Business rules: carton size, whole cartons, mandatory fields, price and delivery rules, no invented values.
- How to work: numbered steps for lookup, one question at a time, saving a draft, confirming, cancelling and escalating.
- Style: short, polite, WhatsApp-friendly messages; at most one bold phrase; no unrelated topics.

Memory

Simple Memory is keyed by the customer's phone number with a short context window. Because memory can carry earlier replies forward (a cause of defect D4), test sessions use a fresh key suffix. Memory resets if n8n restarts.

Model

A Google Gemini chat model with tool calling. A low temperature is recommended for consistent order handling. Early testing used a Groq-hosted open model; the change is recorded in the defect log.

8. Guardrails

Guardrail	How it is enforced
No invented prices or totals	Prices come from get_price_list; totals from the Calculator; prompt forbids guessing
No invented delivery claims	Prompt rule: report only fields present in the orders table (defect D5)
Confirmation before confirming	Prompt step: update_order sets confirmed only after an explicit customer yes
Confirmed orders are not cancelled by the bot	Prompt rule: for a confirmed order, call notify_owner and tell the customer the owner will confirm
Escalation of sensitive cases	Prompt rule plus notify_owner: complaints, discounts, out-of-area delivery, unknown questions
Clean customer messages	Output rule in the prompt, and the reply check node before sending (defect D7)
Language follows the customer	Language rule based on the latest message (defect D4)

9. Error handling

Situation	Behaviour	Improvement for production
Model returns leaked reasoning or overlong text	Reply check substitutes a safe fallback message	Log flagged replies and alert the maintainer
Model rate limit or outage	The execution fails and the customer gets no reply	Fallback model and a paid tier; alert on failed executions
Data table read or write fails	The agent's tool call errors and the run fails	Retry and an owner alert on failure
Owner alert blocked by WhatsApp's 24-hour window	The alert is not delivered	Approved message templates for alerts
Unpublished workflow changes	Old behaviour keeps running	Pre-release checklist (see the Runbook)

10. Security and privacy

- Meta calls the webhook with a verify token that the workflow checks. For production, also validate Meta's request signature.
- Access tokens and model keys live in n8n credentials. The exported workflow and screenshots in the repository contain placeholders only.
- Only the phone number and order details are stored. Test data should be deleted before any real use.
- The owner number is separate from customer numbers and is set in one node.
- Customers should be told they are talking to an automated assistant, and a human fallback should exist.

11. Performance and cost

- Sampled agent executions completed in about 2 to 3.5 seconds.
- Each customer message creates several n8n executions, because Meta also sends delivery and read status updates to the same webhook.

- Cost drivers: n8n plan and execution volume, model usage and rate limits, and WhatsApp conversation charges. Check current pricing before quoting a client.

12. Limitations and path to production

Limitation	Production step
Free or trial tiers and rate limits	Paid model tier and a fallback model
Memory resets on restart	Database-backed memory and conversation logs
Owner alerts depend on the 24-hour window	Approved WhatsApp templates
No signature validation, monitoring or alerts	Validate Meta signatures; monitor failed runs
No payment, stock or delivery data	Integrate payment links, stock and delivery scheduling
Prompt changes are tested by hand	Automated regression tests that replay the 14 scenarios
Occasional stray formatting characters in bold text	Post-process messages for WhatsApp formatting

13. Traceability to requirements

Requirement (BRD v2)	Design element
FR1 Receive and reply on WhatsApp	Webhook, message check, send message nodes
FR2, FR6, FR7 Collect, save and edit a draft	Agent prompt steps; save_pending_order; update_order
FR3 Bottles to cartons	Business rule in the prompt; Calculator
FR4, FR5 Prices and totals	get_price_list; Calculator
FR8, FR9, FR10 Confirm and cancel	update_order; notify_owner; cancellation rule
FR11 Order status	get_my_orders; status rule
FR12 Escalation	notify_owner; escalation rule
FR13 Language	Language rule
FR14 Memory	Simple Memory keyed by phone number
FR15 Output safety	Output rule; reply check node